

Calidad de Diseño y Construcción

Unidad 1: Fundamentos de calidad interna y externa en desarrollo de software



Calidad Externa vs. Calidad Interna

La calidad del software se manifiesta en dos dimensiones fundamentales que, aunque distintas, están profundamente interconectadas. Comprender esta diferencia es esencial para construir sistemas robustos y sostenibles.

Calidad Externa

Lo que el usuario final percibe directamente. Incluye la **eficacia** (que el software haga lo que debe), la **fiabilidad** (que no falle), la **usabilidad** y el **rendimiento**. Se mide con métricas de QA y experiencia de usuario.

Ejemplo: La aplicación funciona correctamente y responde rápidamente a las acciones del usuario.

Calidad Interna

Cómo está construido el código en su interior. Se refiere a la **legibilidad**, el **bajo acoplamiento**, la **alta cohesión** y la **testabilidad**. El usuario no la ve directamente, pero el desarrollador la sufre o la disfruta cada vez que trabaja con el sistema.

Ejemplo: Código claro, modular y fácil de mantener.

Versión 1: Mala Calidad Interna

Este ejemplo muestra una implementación funcional pero con graves problemas de diseño interno. El código funciona correctamente para el usuario (buena calidad externa), pero presenta múltiples defectos de arquitectura que dificultarán su mantenimiento futuro.

```
public class DescuentoService {  
    public double calcular(double total, String tipoCliente) {  
        if (tipoCliente.equals("NORMAL")) {  
            return total;  
        } else if (tipoCliente.equals("VIP")) {  
            return total * 0.9;  
        } else if (tipoCliente.equals("EMPRESA")) {  
            return total * 0.8;  
        } else {  
            return total;  
        }  
    }  
}
```

Strings Hardcodeados

Los tipos de cliente están escritos directamente en el código, lo que dificulta su mantenimiento y hace imposible cambiarlos sin recompilar.

Difícil de Extender

Añadir un nuevo tipo de descuento requiere modificar el método existente, violando el principio de abierto/cerrado.

Uso Excesivo de if-else

La lógica de decisión se implementa con múltiples condiciones anidadas, lo que reduce la legibilidad y aumenta la complejidad ciclomática.

Poca Legibilidad

El desarrollador debe leer toda la estructura condicional para entender qué hace el método, aumentando el tiempo de comprensión.

Versión 2: Buena Calidad Interna

Esta implementación utiliza el patrón Strategy para crear una solución modular, extensible y fácil de mantener. El resultado para el usuario es idéntico, pero la estructura interna es completamente diferente.

```

public interface Descuento {
    double aplicar(double total);
}

public class DescuentoNormal implements Descuento {
    public double aplicar(double total) {
        return total;
    }
}

public class DescuentoVIP implements Descuento {
    public double aplicar(double total) {
        return total * 0.9;
    }
}

public class DescuentoEmpresa implements Descuento {
    public double aplicar(double total) {
        return total * 0.8;
    }
}

import java.util.Map;
public class DescuentoService {
    private Map estrategias;

    public DescuentoService() {
        estrategias = Map.of(
            "NORMAL", new DescuentoNormal(),
            "VIP", new DescuentoVIP(),
            "EMPRESA", new DescuentoEmpresa()
        );
    }

    public double calcular(double total, String tipoCliente) {
        return estrategias
            .getOrDefault(tipoCliente, new DescuentoNormal())
            .aplicar(total);
    }
}
```

01 Código Modular

Cada estrategia de descuento es una clase independiente con una única responsabilidad.

02 Fácil de Extender

Añadir un nuevo tipo de descuento solo requiere crear una nueva clase que implemente la interfaz.

03 Elimina if-else

La lógica de decisión se delega al mapa de estrategias, eliminando las condiciones complejas.

04 Más Legible

El código expresa claramente su intención mediante nombres descriptivos y estructura clara.



Conclusión: Calidad Interna y Externa

Calidad Externa

Ambas versiones ofrecen la misma funcionalidad al usuario. El resultado es idéntico: el descuento se calcula correctamente según el tipo de cliente.

Calidad Interna

La segunda versión tiene significativamente mejor calidad interna, lo que facilita el mantenimiento, la extensión y la escalabilidad del sistema.

- ❏ **Lección clave:** La calidad externa (funcionalidad) y la calidad interna (diseño) son dimensiones independientes. Un sistema puede funcionar perfectamente (buena calidad externa) pero ser un dolor para mantener (mala calidad interna).

Deuda Técnica

La deuda técnica es una metáfora creada por Ward Cunningham que compara el desarrollo de software con las finanzas. Tomar un "atajo" (código sucio o mal diseñado) para entregar rápido genera una deuda que debe pagarse en el futuro.



Atajo Hoy

Entregar rápido sin tests, con código duplicado o diseño deficiente.



Intereses Futuros

Tiempo extra para agregar funcionalidades debido a la mala base.



Costo Total

El costo futuro que pagamos por tomar atajos hoy.

Ejemplos de Deuda Técnica

- **No escribir tests**
Ahorra tiempo ahora, pero genera miedo a modificar el código después.
- **No refactorizar**
Deja el código en estado deficiente, acumulando complejidad.
- **Copiar código**
Duplica la lógica en múltiples lugares, violando el principio DRY.
- **Código confuso**
Usa nombres ambiguos y estructuras difíciles de entender.

Consecuencias de la Deuda Técnica

- **Cambios más lentos**
El desarrollador debe navegar por código complejo y confuso.
- **Más bugs**
La complejidad aumenta la probabilidad de introducir errores.
- **Mayor costo de mantenimiento**
Se necesita más tiempo y recursos para mantener el sistema.

Mantenibilidad del Software

La mantenibilidad es la capacidad del software para ser modificado (corregir errores, mejorar rendimiento o adaptar a cambios) con el menor esfuerzo y riesgo posible. Es un atributo fundamental de calidad interna que determina el costo total de propiedad del sistema.



Entenderse

El código debe ser claro y expresivo para que cualquier desarrollador pueda comprender su funcionamiento sin documentación extensa.



Modificarse

Debe ser posible cambiar el comportamiento del sistema sin romper funcionalidades existentes o introducir nuevos errores.



Extenderse

El diseño debe permitir añadir nuevas características sin modificar código existente, siguiendo el principio abierto/cerrado.



Corregirse

Los errores deben ser fáciles de identificar, localizar y reparar sin efectos colaterales inesperados.

Code Smells: Señales de Problemas de Diseño

El concepto de **code smell** fue creado por Martin Fowler. Un code smell es una señal de que algo en el diseño del código podría estar mal. No es un bug, pero **indica posible problema de diseño**. En un código con muchos smells, el costo del cambio crece de forma **exponencial**.

1

Método Largo

Funciones que intentan realizar demasiadas tareas simultáneamente. Son difíciles de leer, entender y testear de forma aislada.

2

Clase Gigante

Clases que acumulan excesivas responsabilidades, violando el Single Responsibility Principle.

3

Obsesión por Primitivos

Usar tipos básicos para representar conceptos complejos de dominio.

4

Demasiados Parámetros

Métodos con listas extensas de argumentos (5 o más).
Solución: Parameter Object.

5

Código Duplicado

El "enemigo número uno" que viola el principio DRY y genera inconsistencias.

Code Smells en Detalle



A. Método Largo (Long Method)

Descripción: Funciones o métodos que intentan realizar demasiadas tareas simultáneamente.

Problema: Son difíciles de leer, entender y, fundamentalmente, difíciles de testear de forma aislada.

Impacto: El desarrollador "sufre" el código al intentar rastrear la lógica interna.



B. Clase Gigante (Large Class)

Descripción: Clases que acumulan una cantidad excesiva de responsabilidades y líneas de código.

Violación Técnica: Representa una violación directa al **Single Responsibility Principle** (Principio de Responsabilidad Única).



C. Obsesión por Primitivos (Primitive Obsession)

Descripción: Tendencia a usar tipos de datos básicos (como `String`, `int` o `double`) para representar conceptos de dominio complejos.

Ejemplo: Utilizar un simple `String` para manejar un Email en lugar de crear una clase `Email` que contenga su propia lógica de validación.



D. Demasiados Parámetros (Long Parameter List)

Descripción: Métodos que requieren una lista extensa de argumentos (generalmente 5 o más) para ejecutarse.

Solución (Parameter Object): Esta técnica de refactoring agrupa parámetros relacionados en una única clase u objeto.

Ventaja: El método recibe un solo objeto estructurado en lugar de múltiples valores sueltos, mejorando la legibilidad.



E. Código Duplicado (Duplicated Code)

Descripción: Considerado el "enemigo número uno" de la calidad interna.

Principio Violado: Ataca el principio DRY (*Don't Repeat Yourself* - No te repitas).

Riesgo de Mantenimiento: Si una regla de negocio cambia, el desarrollador debe buscar y modificar dicha lógica en múltiples lugares, lo que genera inconsistencias y errores.

Costo del Cambio Exponencial

En un código con muchos *smells*, el costo del cambio crece de forma **exponencial**. Cada atajo tomado y cada problema de diseño ignorado aumenta la complejidad acumulada del sistema.

- 1** — Inicio del Proyecto
Costo bajo de cambio. El código es limpio y la arquitectura es clara.
- 2** — Acumulación de Deuda
Atajos y smells aumentan gradualmente la complejidad del sistema.
- 3** — Punto de Inflexión
El costo de cambio supera el beneficio. El sistema se vuelve rígido.
- 4** — Reescritura Necesaria
El costo de mantener supera el de reconstruir desde cero.

⚠ **Advertencia:** Ignorar los code smells y acumular deuda técnica puede llevar a una crisis de mantenibilidad donde el sistema se vuelve prácticamente imposible de modificar sin introducir nuevos errores.